
Module 03: JavaScript Essentials

Learn the modern JavaScript features you will use every day in Svelte development.

Revolution Trading Pros - Web Dev Course

Overview

JavaScript is the programming language of the web, and modern JavaScript (ES6+) provides powerful features that make your code more concise and expressive. In this module, we cover the essential JavaScript concepts you need for Svelte development. If you are coming from another language, this module will get you up to speed quickly.

Variables: let and const

Modern JavaScript provides two ways to declare variables. The older 'var' keyword has scoping issues and should be avoided.

JavaScript

```
// const - for values that will not be reassigned
const API_URL = "https://api.example.com";
const MAX_RETRIES = 3;
const user = { name: "Alice", age: 30 }; // Object reference is constant
user.age = 31; // This is fine - mutating the object, not reassigning

// let - for values that will change
let count = 0;
count += 1; // Reassignment is allowed

let isLoading = true;
isLoading = false;
```

Rule of thumb: Use const by default. Only use let when you know the value needs to be reassigned. This makes your code more predictable and easier to reason about.

Template Literals

Template literals use backticks instead of quotes and support embedded expressions and multi-line strings:

```
const name = "World";
const greeting = `Hello, ${name}!`; // "Hello, World!"

const price = 29.99;
const tax = 0.1;
const total = `Total: $$${(price * (1 + tax)).toFixed(2)}`; // "Total: $32.99"

// Multi-line strings
const html = `
  <div class="card">
    <h2>${name}</h2>
    <p>Price: $$${price}</p>
  </div>
`;
```

Arrow Functions

Arrow functions provide a concise syntax for writing functions. They also lexically bind 'this', which solves many common JavaScript pitfalls.

JavaScript

```
// Traditional function
function add(a, b) {
  return a + b;
}

// Arrow function
const add = (a, b) => a + b;

// With a body block (for multiple statements)
const processUser = (user) => {
  const fullName = `${user.first} ${user.last}`;
  return { ...user, fullName };
};

// Single parameter - parentheses optional
const double = x => x * 2;

// No parameters - empty parentheses required
const getTimestamp = () => Date.now();
```

Arrow functions are used extensively in Svelte for event handlers, reactive computations, and array transformations.

Array Methods

Array methods are the bread and butter of data manipulation in JavaScript. Svelte templates often use these methods to transform data for display.

map() - Transform Each Element

JavaScript

```
const numbers = [1, 2, 3, 4, 5];
const doubled = numbers.map(n => n * 2); // [2, 4, 6, 8, 10]

const users = [
  { name: "Alice", age: 30 },
  { name: "Bob", age: 25 }
];
const names = users.map(u => u.name); // ["Alice", "Bob"]
```

filter() - Keep Elements That Match

JavaScript

```
const numbers = [1, 2, 3, 4, 5, 6];
const evens = numbers.filter(n => n % 2 === 0); // [2, 4, 6]

const adults = users.filter(u => u.age >= 18);
```

reduce() - Accumulate Into a Single Value

JavaScript

```
const numbers = [1, 2, 3, 4, 5];
const sum = numbers.reduce((acc, n) => acc + n, 0); // 15

// Group items by category
const items = [
  { name: "Apple", category: "fruit" },
  { name: "Carrot", category: "vegetable" },
  { name: "Banana", category: "fruit" }
];

const grouped = items.reduce((acc, item) => {
  const key = item.category;
  acc[key] = acc[key] || [];
  acc[key].push(item);
  return acc;
}, {});
```

Chaining Methods

JavaScript

```
const result = users
  .filter(u => u.age >= 18)
  .map(u => u.name.toUpperCase())
  .sort();
```

Destructuring

Destructuring allows you to extract values from arrays and objects into distinct variables. It is used extensively in Svelte for props, function parameters, and data processing.

Object Destructuring

JavaScript

```
const user = { name: "Alice", age: 30, email: "alice@example.com" };

// Extract specific properties
const { name, age } = user;
console.log(name); // "Alice"
console.log(age);  // 30

// Rename during destructuring
const { name: userName, email: userEmail } = user;

// Default values
const { role = "user" } = user; // "user" (not in original object)

// Nested destructuring
const response = { data: { users: [{ id: 1 }] } };
const { data: { users } } = response;
```

Array Destructuring

JavaScript

```
const colors = ["red", "green", "blue"];
const [first, second] = colors; // "red", "green"
const [, , third] = colors;     // "blue" (skip first two)

// Swap variables
let a = 1, b = 2;
[a, b] = [b, a]; // a = 2, b = 1

// Rest element
const [head, ...tail] = [1, 2, 3, 4]; // head = 1, tail = [2, 3, 4]
```

Spread Operator

The spread operator (...) expands iterables and is essential for immutable data patterns in Svelte.

JavaScript

```
// Spread arrays
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const combined = [...arr1, ...arr2]; // [1, 2, 3, 4, 5, 6]

// Spread objects (shallow copy + merge)
const defaults = { theme: "light", lang: "en", fontSize: 14 };
const prefs = { theme: "dark", fontSize: 16 };
const settings = { ...defaults, ...prefs }; // theme: "dark", lang: "en",
fontSize: 16

// Creating a new object with one property changed (immutable update)
const updatedUser = { ...user, age: 31 };
```

Async/Await

Modern JavaScript handles asynchronous operations with Promises and the `async/await` syntax. This is essential for API calls, file operations, and any task that takes time.

JavaScript

```
// Fetching data with async/await
async function fetchUsers() {
  try {
    const response = await fetch("https://api.example.com/users");

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const users = await response.json();
    return users;
  } catch (error) {
    console.error("Failed to fetch users:", error);
    throw error; // Re-throw to let the caller handle it
  }
}

// Using the function
const users = await fetchUsers();
console.log(users);
```

Parallel Requests

JavaScript

```
// Run multiple requests in parallel
const [users, posts, comments] = await Promise.all([
  fetch("/api/users").then(r => r.json()),
  fetch("/api/posts").then(r => r.json()),
  fetch("/api/comments").then(r => r.json())
]);
```

ES Modules

ES Modules are the standard module system for JavaScript. SvelteKit uses ES Modules throughout.

TypeScript

```
// utils.ts - Named exports
export function formatDate(date: Date): string {
  return date.toLocaleDateString();
}

export function formatCurrency(amount: number): string {
  return `$$${amount.toFixed(2)}`;
}

export const API_BASE = "https://api.example.com";

// Default export (one per module)
export default class ApiClient {
  // ...
}
```

TypeScript

```
// Importing in another file
import ApiClient, { formatDate, formatCurrency, API_BASE } from "./utils";

// Import everything as a namespace
import * as utils from "./utils";
utils.formatDate(new Date());

// SvelteKit $lib alias
import Header from "$lib/components/Header.svelte";
import { formatDate } from "$lib/utils";
```

Module 3 Exercise

Exercise: Data-Driven List

Put what you have learned into practice with this hands-on exercise.

- %İ Step 1: Create a new file `src/lib/data/products.ts` with an array of 10 product objects, each having `id`, `name`, `price`, `category`, and `inStock` properties.
- %İ Step 2: Export the array as a named export.
- %İ Step 3: In `src/routes/+page.svelte`, import the `products` array.
- %İ Step 4: Display all products using `{#each}` with destructuring.
- %İ Step 5: Add filter buttons for each category using `array.filter()`.
- %İ Step 6: Display the total price of displayed products using `array.reduce()`.
- %İ Step 7: Add a search input that filters products by name using `string.toLowerCase().includes()`.
- %İ Step 8: Commit your changes to Git.

Summary

- %İ Use `const` by default, `let` when reassignment is needed. Avoid `var`.
- %İ Template literals make string interpolation clean and readable.
- %İ Arrow functions are concise and solve 'this' binding issues.
- %İ `map`, `filter`, and `reduce` are essential for data transformation.
- %İ Destructuring extracts values from objects and arrays concisely.
- %İ The spread operator enables immutable data patterns.
- %İ `async/await` makes asynchronous code readable and maintainable.
- %İ ES Modules organize code into reusable, importable pieces.